

Table of Contents

- 1. SMC 2
 - 1.1. Per-Core Reset 2
 - 1.2. External CPU cluster NDM reset 2
- 2. SEP 3
 - 2.1. CPU Bring-Up 3
 - 2.1.1. NMI Vector Programming 3
 - 2.2. OCAH TRNG Bring-Up 3
 - 2.2.1. Bring-up sequence at a glance 4
 - 2.2.2. Internal entropy-chain initialization 5
 - 2.2.3. Failure handling and recovery 6
- 3. AXI-over-UCIe Bridge (AoU) Software Operation 7
- Appendix C. Software operation guide 7
 - SFR setting 7
 - Debugging Features 13
 - Activation & Deactivation Flow 16

WARNING

This documentation product is currently under development. More content will be added as the project matures.

Chapter 1. SMC

1.1. Per-Core Reset

See the *SMC CPU Reset, Boot, and Initialization* section of the Technical Reference Manual for the hardware behavior and associated risks.

WARNING | Do not reset a hart while any hart is performing coherent memory accesses.

A safe startup scenario, subject to the warning above, is when three of four harts remain parked after ROM boot and must start at a different firmware address. Before they join normal coherent operation, software can program a parked hart's `RESET_VECTOR[n]` and pulse its reset to start it at that address. Avoid this flow where possible for the reasons described in the Technical Reference Manual.

1.2. External CPU cluster NDM reset

Read `NDMRESET_REQUEST` bit `i` as the NDM reset request from external CPU cluster `i`. Write `NDMRESET_PROCESS` bit `i` to answer that cluster. Read `NDMRESET_CLUSTER_COUNT` and mask both registers with that value.

This harness reports 4 in `NDMRESET_CLUSTER_COUNT`. Bits of `NDMRESET_REQUEST` at and above the count read as 0. Writes to those bits of `NDMRESET_PROCESS` stay in the register.

The register block is in the *Chip Config, Scratch, NDM Reset, and Misc Wrap* section of the Technical Reference Manual.

Chapter 2. SEP

2.1. CPU Bring-Up

2.1.1. NMI Vector Programming

The Security Processor (SEP) runs a VeeR-EL2 core whose Non-Maskable Interrupt is driven by the watchdog timer bark (`sep.sv` wires `nmi_int` to `intr_wdog_timer_bark`). When the NMI asserts, the core jumps to the address held in the `SEP_NMI_VEC` register of the `sep_cpu_ctrl` block (offset `0x180`).

Software reads and writes `SEP_NMI_VEC` as a byte address. The register's `nmi_vec` field occupies bits 31:1, so bit 0 is reserved and always reads back as zero; the hardware imposes no stricter alignment than that. The reference DV firmware nevertheless aligns its NMI trampoline to 256 bytes, which the linker script enforces.

WARNING

The `SEP_NMI_VEC` reset value (`0xC000_0100`) is a placeholder. It does not point at a valid handler, and no part of the boot flow makes it one. Firmware must complete the following sequence before enabling the watchdog timer or any other NMI source:

1. Place an NMI handler in ICCM.
2. Write the handler's address to `SEP_NMI_VEC`.
3. Set `SEP_NMI_VEC_LOCK` (offset `0x188`) to freeze the vector until the next reset.
4. Only then enable the WDT.

If a bark arrives while the reset value is still in place, the core jumps to `0xC000_0100` and executes whatever occupies that location: arbitrary instructions if the loaded image covers it, or an uncorrectable ICCM ECC error if it does not. The ECC case re-raises NMI, so the core makes no forward progress.

Because `SEP_NMI_VEC_LOCK` is write-one-to-set and sticky until reset, the vector cannot be repointed once locked. Firmware that needs to install a different handler after lock must do so through its own indirection — the reference firmware keeps a runtime-writable handler pointer that the trampoline dispatches through, so the locked vector always targets the trampoline.

If the watchdog bite threshold is also enabled, an unserviced bark still escalates to the watchdog reset path and resets the SEP. A mis-programmed vector therefore degrades to a watchdog reset rather than a silent hang, but it does so only after the bite interval and without running any handler.

2.2. OCAH TRNG Bring-Up

SEP has three entropy streams: Key Manager, the AES/KMAC/OTBN group, and the fabric entropy pool. `SEP_CPU_CTRL.EXT_TRNG_SRC_SEL` selects the source of each stream independently:

- bit 0 selects the Key Manager stream;
- bit 1 selects the AES/KMAC/OTBN stream; and
- bit 2 selects the fabric entropy-pool stream.

A bit value of 0 selects the internal ESRC → CSRNG → EDN chain. A bit value of 1 selects the corresponding external

TRNG stream and bypasses the internal chain for that stream.

WARNING

The OCAH internal entropy source has not been certified. Consequently, `EXT_TRNG_SRC_SEL` resets to `0x7`, selecting the external TRNG for all three streams. Production firmware should retain that default and use an appropriately qualified external source unless the product security authority explicitly approves the OCAH internal path.

The CSRNG and EDN blocks are derived from OpenTitan. OpenTitan describes CSRNG as compatible with NIST SP 800-90A and AIS31 and implements the approved CTR_DRBG construction, but this is not a certification of the OCAH integration. NIST CAVP validation applies to a specific implementation and operational environment. No CAVP or FIPS 140 validation is claimed for the OCAH entropy chain. See the [OpenTitan CSRNG specification](#) and [NIST CAVP](#).

`EXT_TRNG_SRC_SEL_LOCK`.LOCK makes the selection read-only until SEP reset. Production firmware should lock the selection after it has verified that the chosen source is operational. An external-TRNG integration must initialize and test that device through its platform-specific control interface; the OCAH sequence below applies only to streams firmware explicitly changes to 0.

2.2.1. Bring-up sequence at a glance

```
Failed to generate image: PlantUML image generation failed: Cannot invoke
"java.lang.reflect.Method.invoke(Object, Object[])" because
"org.asciidoctor.diagram.plantuml.PlantUML.SET_LOCAL_IMAGE_LIMIT" is null
@startuml
start
:Stop dispatching new entropy-dependent operations;
:Keep the Key Manager idle;
:Choose and initialize entropy source;
if (Use OCAH TRNG?) then (yes)
    :Pulse shared TRNG reset;
    :Set required EXT_TRNG_SRC_SEL bits to 0;
    :Configure ESRC with generators off;
    :Enable CSRNG and configure EDN auto mode;
    :Enable approved ring oscillators;
    if (ESRC boot health test passes?) then (yes)
        :Apply FIPS_LOCK if required;
        :Enable EDN and issue Instantiate;
        if (Instantiate acknowledged successfully?) then (yes)
        else (no)
            :Record CSRNG/EDN status;
            :Fail initialization;
        stop
    endif
else (no)
    :Record ESRC health and alert status;
    :Fail initialization;
    stop
endif
```

```

else (external)
    :Keep required EXT_TRNG_SRC_SEL bits at 1;
    :Validate the external TRNG;
endif
:Lock source selection if required;
:Resume entropy-dependent operations;
stop
@enduml

```

2.2.2. Internal entropy-chain initialization

Use generated register definitions and named fields rather than copying complete register literals. Several control fields use MuBi4 encodings, for which true is 0x6 and false is 0x9.

1. Prevent new entropy-dependent operations during initialization. On first boot, initialize entropy before accepting crypto work; the Key Manager is held in software reset, which keeps it idle. A runtime recovery must wait until KM is idle, or hold it in software reset, before pulsing the TRNG reset. AES, KMAC, and OTBN may stall until fresh entropy is available, or, after each is idle, firmware may hold them in software reset to abort their operations.
2. Reset the internal entropy chain by clearing and then setting `SEP_RESET_CTRL.SW_RESET_N.trng_sw_rst_n` with read-modify-write operations that preserve the other reset bits. This reset covers ESRC, CSRNG, EDN, their control-plane adapters, buffered internal entropy, and the fabric entropy pool. `ESRC.CTRL[0]` is reserved and is not a reset control.
3. Select the internal source by clearing each required bit in `EXT_TRNG_SRC_SEL`. This mux programming is a required part of OCAH TRNG initialization; the reset value deliberately continues to select the external source.
4. Disable `ESRC.RING_OSC_ENABLE.ENABLE` while programming the entropy-source policy. Program the approved sampling, health-test, FIFO, conditioning, and module-enable fields. In particular, the intended standards-aligned profile requires `CTRL.MODULE_ENABLE=1`, the approved health-test thresholds and window, entropy-compressor bypass disabled, and SHA-256 whitening enabled.
5. Enable the CSRNG with `CSRNG.CTRL.ENABLE`. Leave `SW_APP_ENABLE`, `READ_INT_STATE`, and `FIPS_FORCE_ENABLE` false unless the security policy explicitly requires those diagnostic interfaces.
6. Configure EDN continuous-operation mode while EDN is disabled:
 - program `RESEED_CMD` and `GENERATE_CMD` with commands that request entropy from CSRNG;
 - program `MAX_NUM_REQS_BETWEEN_RESEEDS`; and
 - set `BOOT_REQ_MODE` false and `AUTO_REQ_MODE` true.
7. Enable the approved ring oscillators and sample clocks in `ESRC.RING_OSC_ENABLE`.
8. Poll `ESRC.MAIN_SM_STATUS.BOOT_PHASE_DONE` with a bounded timeout. While waiting, fail initialization if `MAIN_SM_STATUS.ALERT`, `MAIN_SM_STATUS.ERR`, or `INTR_STATUS.PERSISTENT_FAILURE` asserts. `BOOT_PHASE_DONE` proves that the startup health-test gate opened; it does not by itself prove that CSRNG accepted a complete seed.
9. If required by the security policy, set `ESRC.FIPS_LOCK.LOCK` after startup passes and before exposing entropy to consumers. The lock freezes the certified ESRC configuration until the internal entropy chain is reset. `DEBUG_CTRL` intentionally remains outside this lock; the platform must separately restrict any externally routed debug-observation signal.
10. Enable EDN with `EDN_ENABLE=true`, `AUTO_REQ_MODE=true`, `BOOT_REQ_MODE=false`, and `CMD_FIFO_RST=false`.
11. Wait for both `EDN.SW_CMD_STS.CMD_RDY` and `CMD_REG_RDY`, then write an Instantiate command with no additional data to `EDN.SW_CMD_REQ`. Poll `CMD_ACK` or `INTR_STATE.EDN_CMD_REQ_DONE` with a bounded timeout and require `CMD_STS` to report

success.

12. Lock `EXT_TRNG_SRC_SEL` if required, then release or initialize the entropy consumers.

The EDN Instantiate acknowledgement is the software-visible readiness event: CSRNG cannot acknowledge that command until it has accepted a complete ESRC seed. Firmware therefore does not need a delay or a hierarchical `drbg_seed_valid` probe. `BOOT_PHASE_DONE` remains useful for separating an ESRC startup-health failure from a downstream CSRNG or EDN failure.

WARNING

Do not set EDN boot-request and auto-request mode true at the same time. Boot-request mode takes precedence and remains in its completed state until firmware clears it; merely setting both modes does not transition EDN to continuous auto-request operation.

`SEP_CPU_CTRL.CLOCK_GATE_CTRL` currently has no entropy clock-gate field. Firmware must not write an undocumented bit 10 as part of entropy bring-up.

2.2.3. Failure handling and recovery

Treat entropy initialization as a fallible operation. Every poll must have a platform-defined timeout, and firmware should preserve enough status to distinguish:

- ESRC startup-health failure (`MAIN_SM_STATUS`, `HEALTH_TEST_STATUS`, and `INTR_STATUS`);
- CSRNG command, exception, recoverable-alert, or fatal-alert failure; and
- EDN command, recoverable-alert, or fatal-alert failure.

The boot policy must define whether any such failure stops secure boot or enters a platform recovery flow. Continuing into an entropy-dependent operation is not recovery: the consumer can wait indefinitely for entropy.

For a runtime restart of the internal chain:

1. Stop dispatching new entropy-dependent operations. The Key Manager must be idle before the TRNG reset is asserted: wait until it is not consuming entropy, or hold it in software reset. If policy requires aborting AES, KMAC, or OTBN in-flight work, wait for each to become idle, save `SW_RESET_N`, and hold those consumers in reset.
2. Clear `trng_sw_rst_n`; hardware drains accepted control transactions, isolates the three internal TRNG register ports, and clears buffered internal and post-mux entropy before asserting reset.
3. Set `trng_sw_rst_n`.
4. Repeat the complete internal initialization sequence, including the bounded ESRC startup poll and acknowledged EDN Instantiate command.
5. Restore any consumer-reset state saved in the first step after initialization succeeds.

The native EDN boundary uses request/acknowledge, not valid/ready. During TRNG reset the post-mux adapters suppress acknowledgements and discard buffered entropy. A request already asserted by AES, KMAC, or OTBN remains asserted and is serviced with fresh entropy after the adapter and internal chain restart. Resetting those consumers is an operation-abort policy, not a prerequisite for the coordinated TRNG reset.

The Key Manager AXI-Stream has no post-mux adapter, so TRNG reset does not scrub it. A reset while the DRBG sampler is requesting or assembling a word can mix leftover bytes into the next DATA read or raise a stream error. The Key Manager must be idle before the SEP host asserts the TRNG reset.

Do not try to recover only ESRC with `CTRL[0]`. A shared TRNG reset invalidates the entire internal chain.

Chapter 3. AXI-over-UCIe Bridge (AoU)

Software Operation

Appendix C. Software operation guide

This section gives information about operation guide for the AOU_CORE.

SFR setting

Register	Offset	Bit field name	Bit field	Type	Reset value
IP_VERSION	0x0000				0x00010000
		MAJOR_VERSION	[31:16]	RO	0x0001
		MINOR_VERSION	[15:0]	RO	0x0000
AOU_CON0	0x0004				0x00000000
		Rsvd	[31:28]	RO	0x0
		RP3_ERROR_INFO_ACCESS_EN	[27]	RW	0x0
		RP2_ERROR_INFO_ACCESS_EN	[26]	RW	0x0
		RP1_ERROR_INFO_ACCESS_EN	[25]	RW	0x0
		RP0_ERROR_INFO_ACCESS_EN	[24]	RW	0x0
		RP3_AXI_AGGREGATOR_EN	[23]	RW	0x0
		RP2_AXI_AGGREGATOR_EN	[22]	RW	0x0
		RP1_AXI_AGGREGATOR_EN	[21]	RW	0x0
		RP0_AXI_AGGREGATOR_EN	[20]	RW	0x0
		TX_LP_MODE_THRESHOLD	[19:12]	RW	0x4
		TX_LP_MODE	[11]	RW	0x0
		Rsvd	[10:5]	RO	0x0
		AOU_SW_RESET	[4]	RW	0x0
		CREDIT_MANAGE	[3]	RW	0x0
		AXI_SPLIT_TR_EN	[2]	RW	0x0
		Rsvd	[1:0]	RO	0x0
AOU_INIT	0x0008				0x00004608
		Rsvd	[31:11]	RO	0x0
		MST_TR_COMPLETE	[10]	RO	0x1
		SLV_TR_COMPLETE	[9]	RO	0x1
		INT_ACTIVATE_START	[8]	W1C	0x0
		INT_DEACTIVATE_START	[7]	W1C	0x0
		DEACTIVATE_TIME_OUT_VALUE	[6:4]	RW	0x0
		ACTIVATE_STATE_DISABLED	[3]	RO	0x1
		ACTIVATE_STATE_ENABLED	[2]	RO	0x0

Register	Offset	Bit field name	Bit field	Type	Reset value
		DEACTIVATE_START	[1]	RW	0x0
		ACTIVATE_START	[0]	RW	0x0
AOU_INTERRUPT_MASK	0x000C				
		Rsvd	[31:9]	RO	0x0
		INT_REQ_LINKRESET_ACT_ACK_MASK	[8]	RW	0x0
		INT_REQ_LINKRESET_DEACT_ACK_MASK	[7]	RW	0x0
		INT_REQ_LINKRESET_INVALID_ACTMSG_MASK	[6]	RW	0x0
		INT_REQ_LINKRESET_MSGCREDIT_TIMEOUT_MASK	[5]	RW	0x0
		INT_EARLY_RESP_MASK	[4]	RW	0x0
		INT_MI0_ID_MISMATCH_MASK	[3]	RW	0x0
		INT_SI0_ID_MISMATCH_MASK	[2]	RW	0x0
		Rsvd	[1]	RO	0x0
		Rsvd	[0]	RO	0x0
LP_LINKRESET	0x0010				
		Rsvd	[31:14]	RO	0x0
		ACK_TIME_OUT_VALUE	[13:11]	RW	0x4
		MSGCREDIT_TIME_OUT_VALUE	[10:8]	RW	0x4
		ACT_ACK_ERR	[7]	W1C	0x0
		DEACT_ACK_ERR	[6]	W1C	0x0
		INVALID_ACTMSG_INFO	[5:2]	RO	0x0
		INVALID_ACTMSG_ERR	[1]	W1C	0x0
		MSGCREDIT_ERR	[0]	W1C	0x0
DEST_RP	0x0014				0x00000000
		Rsvd	[31:14]	RO	0x0
		RP3_DEST	[13:12]	RW	0x3
		Rsvd	[11:10]	RO	0x0
		RP2_DEST	[9:8]	RW	0x2
		Rsvd	[7:6]	RO	0x0
		RP1_DEST	[5:4]	RW	0x1
		Rsvd	[3:2]	RO	0x0
		RP0_DEST	[1:0]	RW	0x0
PRIOR_RP_AXI	0x0018				
		Rsvd	[31:28]	RO	0x0
		AXI_QOS_TO_NP	[27:24]	RW	0xA
		AXI_QOS_TO_HP	[23:20]	RW	0x5
		Rsvd	[19:18]	RO	0x0
		RP3_PRIOR	[17:16]	RW	0x3
		Rsvd	[15:14]	RO	0x0

Register	Offset	Bit field name	Bit field	Type	Reset value
		RP2_PRIOR	[13:12]	RW	0x2
		Rsvd	[11:10]	RO	0x0
		RP1_PRIOR	[9:8]	RW	0x1
		Rsvd	[7:6]	RO	0x0
		RP0_PRIOR	[5:4]	RW	0x0
		Rsvd	[3:2]	RO	0x0
		ARB_MODE	[1:0]	RW	0x0
PRIOR_TIMER	0x001C				0x00000000
		TIMER_RESOLUTION	[31:16]	RW	0x0
		TIMER_THRESHOLD	[15:0]	RW	0x0
AXI_SPLIT_TR_RP0	0x0020				0x00000F0F
		Rsvd	[31:16]	RO	0x0
		MAX_AWBURSTLEN	[15:8]	RW	0xF
		MAX_ARBURSTLEN	[7:0]	RW	0xF
ERROR_INFO_RP0	0x0024				0x0
		Rsvd	[31:22]	RO	0x0
		SPLIT_BID_MISMATCH_INFO	[21:12]	RO	0x0
		SPLIT_RID_MISMATCH_INFO	[11:2]	RO	0x0
		SPLIT_BID_MISMATCH_ERR	[1]	W1C	0x0
		SPLIT_RID_MISMATCH_ERR	[0]	W1C	0x0
WRITE_EARLY_RESPONSE_RP0	0x0028				0x0
		Rsvd	[31:15]	RO	0x0
		WRITE_RESP_DONE	[14]	RO	0x0
		WRITE_RESP_ERR	[13]	W1C	0x0
		WRITE_RESP_ERR_TYPE_INFO	[12:11]	RO	0x0
		WRITE_RESP_ERR_ID_INFO	[10:1]	RO	0x0
		EARLY_BRESP_EN	[0]	RW	0x0
AXI_ERROR_INFO0_RP0	0x002C				0x0
		DEBUG_UPPER_ADDR	[31:0]	RW	0x0
AXI_ERROR_INFO1_RP0	0x0030				0x0
		DEBUG_LOWER_ADDR	[31:0]	RW	0x0
AXI_SLV_ID_MISMATCH_ERR_RP0	0x0034				
		Rsvd	[31:22]	RO	0x0
		AXI_SLV_BID_MISMATCH_INFO	[21:12]	RO	0x0
		AXI_SLV_RID_MISMATCH_INFO	[11:2]	RO	0x0
		AXI_SLV_BID_MISMATCH_ERR	[1]	W1C	0x0
		AXI_SLV_RID_MISMATCH_ERR	[0]	W1C	0x0
AXI_SPLIT_TR_RP1	0x0038				0x00000F0F

Register	Offset	Bit field name	Bit field	Type	Reset value
		Rsvd	[31:16]	RO	0x0
		MAX_AWBURSTLEN	[15:8]	RW	0xF
		MAX_ARBURSTLEN	[7:0]	RW	0xF
ERROR_INFO_RP1	0x003C				0x0
		Rsvd	[31:22]	RO	0x0
		SPLIT_BID_MISMATCH_INFO	[21:12]	RO	0x0
		SPLIT_RID_MISMATCH_INFO	[11:2]	RO	0x0
		SPLIT_BID_MISMATCH_ERR	[1]	W1C	0x0
		SPLIT_RID_MISMATCH_ERR	[0]	W1C	0x0
WRITE_EARLY_RESPONSE_RP1	0x0040				0x0
		Rsvd	[31:15]	RO	0x0
		WRITE_RESP_DONE	[14]	RO	0x0
		WRITE_RESP_ERR	[13]	W1C	0x0
		WRITE_RESP_ERR_TYPE_INFO	[12:11]	RO	0x0
		WRITE_RESP_ERR_ID_INFO	[10:1]	RO	0x0
		EARLY_BRESP_EN	[0]	RW	0x0
AXI_ERROR_INFO0_RP1	0x0044				0x0
		DEBUG_UPPER_ADDR	[31:0]	RW	0x0
AXI_ERROR_INFO1_RP1	0x0048				0x0
		DEBUG_LOWER_ADDR	[31:0]	RW	0x0
AXI_SLV_ID_MISMATCH_ERR_RP1	0x004C				
		Rsvd	[31:22]	RO	0x0
		AXI_SLV_BID_MISMATCH_INFO	[21:12]	RO	0x0
		AXI_SLV_RID_MISMATCH_INFO	[11:2]	RO	0x0
		AXI_SLV_BID_MISMATCH_ERR	[1]	W1C	0x0
		AXI_SLV_RID_MISMATCH_ERR	[0]	W1C	0x0
AXI_SPLIT_TR_RP2	0x0050				0x00000F0F
		Rsvd	[31:16]	RO	0x0
		MAX_AWBURSTLEN	[15:8]	RW	0xF
		MAX_ARBURSTLEN	[7:0]	RW	0xF
ERROR_INFO_RP2	0x0054				0x0
		Rsvd	[31:22]	RO	0x0
		SPLIT_BID_MISMATCH_INFO	[21:12]	RO	0x0
		SPLIT_RID_MISMATCH_INFO	[11:2]	RO	0x0
		SPLIT_BID_MISMATCH_ERR	[1]	W1C	0x0
		SPLIT_RID_MISMATCH_ERR	[0]	W1C	0x0
WRITE_EARLY_RESPONSE_RP2	0x0058				0x0
		Rsvd	[31:15]	RO	0x0

Register	Offset	Bit field name	Bit field	Type	Reset value
		WRITE_RESP_DONE	[14]	RO	0x0
		WRITE_RESP_ERR	[13]	W1C	0x0
		WRITE_RESP_ERR_TYPE_INFO	[12:11]	RO	0x0
		WRITE_RESP_ERR_ID_INFO	[10:1]	RO	0x0
		EARLY_BRESP_EN	[0]	RW	0x0
AXI_ERROR_INFO0_RP2	0x005C				0x0
		DEBUG_UPPER_ADDR	[31:0]	RW	0x0
AXI_ERROR_INFO1_RP2	0x0060				0x0
		DEBUG_LOWER_ADDR	[31:0]	RW	0x0
AXI_SLV_ID_MISMATCH_ERR_RP2	0x0064				
		Rsvd	[31:22]	RO	0x0
		AXI_SLV_BID_MISMATCH_INFO	[21:12]	RO	0x0
		AXI_SLV_RID_MISMATCH_INFO	[11:2]	RO	0x0
		AXI_SLV_BID_MISMATCH_ERR	[1]	W1C	0x0
		AXI_SLV_RID_MISMATCH_ERR	[0]	W1C	0x0
AXI_SPLIT_TR_RP3q	0x0068				0x00000F0F
		Rsvd	[31:16]	RO	0x0
		MAX_AWBURSTLEN	[15:8]	RW	0xF
		MAX_ARBURSTLEN	[7:0]	RW	0xF
ERROR_INFO_RP3	0x006C				0x0
		Rsvd	[31:22]	RO	0x0
		SPLIT_BID_MISMATCH_INFO	[21:12]	RO	0x0
		SPLIT_RID_MISMATCH_INFO	[11:2]	RO	0x0
		SPLIT_BID_MISMATCH_ERR	[1]	W1C	0x0
		SPLIT_RID_MISMATCH_ERR	[0]	W1C	0x0
WRITE_EARLY_RESPONSE_RP3	0x0070				0x0
		Rsvd	[31:15]	RO	0x0
		WRITE_RESP_DONE	[14]	RO	0x0
		WRITE_RESP_ERR	[13]	W1C	0x0
		WRITE_RESP_ERR_TYPE_INFO	[12:11]	RO	0x0
		WRITE_RESP_ERR_ID_INFO	[10:1]	RO	0x0
		EARLY_BRESP_EN	[0]	RW	0x0
AXI_ERROR_INFO0_RP3	0x0074				0x0
		DEBUG_UPPER_ADDR	[31:0]	RW	0x0
AXI_ERROR_INFO1_RP3	0x0078				0x0
		DEBUG_LOWER_ADDR	[31:0]	RW	0x0
AXI_SLV_ID_MISMATCH_ERR_RP3	0x007C				
		Rsvd	[31:22]	RO	0x0

Register	Offset	Bit field name	Bit field	Type	Reset value
		AXI_SLV_BID_MISMATCH_INFO	[21:12]	RO	0x0
		AXI_SLV_RID_MISMATCH_INFO	[11:2]	RO	0x0
		AXI_SLV_BID_MISMATCH_ERR	[1]	W1C	0x0
		AXI_SLV_RID_MISMATCH_ERR	[0]	W1C	0x0

Table 7. Register Map

ERR SET is restricted to change the value while the system is running

For details on the ACTIVATE and DEACTIVATE SFRs, please refer to [Activation & Deactivation Flow](#).

AOU_INIT.ACTIVATE_START

- After link-up is completed, AOU_CORE can send or receive an ActivateReq message.
- AoU Activation Request can only be triggered by AOU_INIT.ACTIVATE_START, and it is allowed to be set before and after the completion of the UCle Link-up.

AOU_INIT.DEACTIVATE_START

- AOU_CORE must exchange both DeactivateReq and DeactivateAck messages before the UCle Core transitions to the Link Down state.
- The DEACTIVATE_START field in AOU_INIT initiates deactivation and ensures that a DeactivateReq is issued only after both conditions below are satisfied:
 1. All outstanding responses to remote die requests have been completed.
 2. No additional messages remain to be transmitted within a window defined by **TIME_OUT_VALUE**.
- **AOU_INIT.DEACTIVATE_TIME_OUT_VALUE** configures the timeout value used for this condition. This field is 3 bits wide and encodes a timeout of $2^{(\text{DEACTIVATE_TIME_OUT_VALUE} + 3)}$ cycles.

MAX BURSTLEN

- The **MAX_AWBURSTLEN** and **MAX_ARBURSTLEN** register fields define the maximum burst length that the splitter may issue to the local bus.
- The splitter inside **AOU_CORE** issues AXI transfers with burst lengths less than or equal to the configured values (**MAX_AWBURSTLEN**, **MAX_ARBURSTLEN**).
- If **AOU_CORE** receives a larger burst length from the remote die, the splitter divides it into smaller bursts based on **MAX_AxBURSTLEN**.
- Since Splitter only supports maximum burst lengths in the form of 2^n , the field must be configured with one of the following: 1, 3, 5, 7, 15, 31, 63, 127 or 255.

ACTIVATE_STATE_ENABLED/DISABLED

- This signal indicates the activation status of the AOU_ACTIVATION_CTRL.
- The ENABLE signal is asserted to indicate that activation has been completed.

The DISABLED signal is asserted to indicate that deactivation has been completed.

MST_TR_COMPLETE/SLV_TR_COMPLETE

- This signal indicates that there are no pending transactions in both local die and remote die.

ACK_TIME_OUT_VALUE/MSG_CREDIT_TIME_OUT_VALUE

- After sending an AoU Activation/Deactivation Request message, if an Ack message is not received within the specified ACK_TIME_OUT_VALUE, a LinkReset interrupt is sent to the SW.
- After receiving the Activation Ack message from the remote die, if a creditgrant message is not received within the specified MSGCREDIT_TIME_OUT_VALUE, a LinkReset interrupt is sent to the SW.
- These fields are 3 bits wide and encode a timeout of $2^{(TIME_OUT_VALUE + 18)}$ core clock cycles.

WRITE_EARLY_RESPONSE

- For the EARLY_RESPONSE_EN SFR setting to enable or disable early response, there must be no pending write requests.
- EARLY_RESPONSE_EN: If this field is set(1), It performs the early response function.
- WRITE_RESP_DONE: It indicates that there are no pending write requests and all write responses have been received from the remote die.
- WRITE_RESP_ERR: It indicated that although the early response feature was enabled and an early response has already been delivered to the local die, an error occurred later in the write response received from the remote die. The BRESP value of the error is stored in WRITE_RESP_ERR_TYPE_INFO, and the BID of the transaction that caused the error is stored in WRITE_RESP_ERR_ID_INFO SFR. When an error occurs, an interrupt is generated. Afterward, writing 1 to the WRITE_RESP_ERR SFR clears ERR_TYPE_INFO, ERR_ID_INFO, and RESP_ERR.

Low Power mode

- For lowering power, AOU_CORE uses low power mode. By setting AOU_CON0.TX_LP_MODE = 1, AOU_CORE only sends payload to FDI when there are remaining AXI messages to send. Since AOU_CORE need to send Credit when there are no remaining AXI messages, by setting AOU_CON0.TX_LP_MODE_THRESHOLD value, SW can configure the message transmission frequency.
- AOU_CON0.TX_LP_MODE_THRESHOLD = 0 selects a fully-gated low-power mode: the periodic heartbeat is disabled and the TX only launches an FDI flit when there is real work to send (AXI traffic, CrdtGrant credit-return messages, or pending MsgCredit-header credits). Once a flit is started it always finishes; any granule slots that were not filled are sent as zeros and the corresponding MsgStart bits are 0.

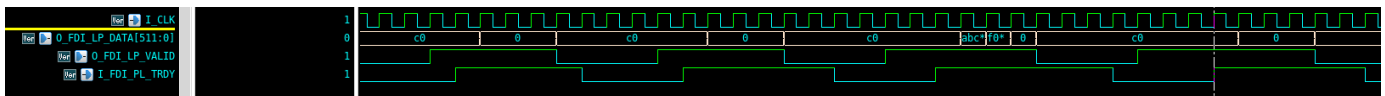


Figure 22. Bubble inserted when Transmitter de-assert LP_VALID

- Default value is 0. AOU_CORE always sends messages to FDI for lowering latency

Debugging Features

Overview

AOU_CORE provides several error-detection and debug features, each with a dedicated interrupt output and a per-feature mask bit in the AOU_INTERRUPT_MASK register (0x000C). The table below summarises the mask bits and corresponding interrupt signals.

Mask Bit	Feature	Interrupt Signal
[8]	Activate-ack timeout	INT_REQ_LINKRESET
[7]	Deactivate-ack timeout	INT_REQ_LINKRESET
[6]	Invalid ACTMSG	INT_REQ_LINKRESET
[5]	MsgCredit timeout	INT_REQ_LINKRESET
[4]	Early write-response error	INT_EARLY_RESP_ERR
[3]	Manager ID mismatch	INT_MIO_ID_MISMATCH
[2]	Subordinate ID mismatch	INT_SIO_ID_MISMATCH

Setting a mask bit to 1 masks (disables) the corresponding interrupt.

AXI ID Mismatch Error

AOU_CORE can detect AXI ID mismatches on both the manager and subordinate interfaces. A mismatch occurs when a B or R response arrives with an AXI ID that does not correspond to any outstanding request.

Manager side. The split/aggregator path (AOU_AXI_SPLIT_TR / AOU_AXI4MUX_3X1_TOP) maintains pending-ID tables for both read and write transactions (AOU_SPLIT_RD_PENDING_INFO and AOU_SPLIT_WR_PENDING_INFO). When a B response arrives with an ID that does not match any pending write entry, SPLIT_BID_MISMATCH_ERR is asserted and the offending ID is captured in SPLIT_BID_MISMATCH_INFO. The same logic applies to R responses via SPLIT_RID_MISMATCH_ERR and SPLIT_RID_MISMATCH_INFO. These fields are in the per-RP ERROR_INFO_RPn register (RP0 at 0x0024, RP1 at 0x003C, RP2 at 0x0054, RP3 at 0x006C). SW clears each error by writing 1 to the corresponding W1C bit. The aggregate interrupt is INT_MIO_ID_MISMATCH.

Table depth and backpressure. Each pending-ID table is sized by M_RD_MO_CNT / M_WR_MO_CNT (default 32), matching the manager interface outstanding transaction limit. An entry is allocated on the split-AR or split-AW handshake and freed when the corresponding R (with RLAST) or B response completes. When all entries are occupied, the O_AR_Slot_Available_Flag / O_AW_Slot_Available_Flag signal goes low, which suppresses new split transactions and effectively deasserts ARREADY / AWREADY toward the upstream path. This is backpressure, not an error – no entries are overwritten or dropped. Because the table depth equals the AXI outstanding count, the table should never overflow during correct operation.

No disable mechanism. There is no RTL parameter or SFR to disable the pending-ID tables or the mismatch check. The interrupt can be masked via AOU_INTERRUPT_MASK.INT_MIO_ID_MISMATCH_MASK [3], but this only suppresses the interrupt signal. The tables continue to operate, backpressure still applies when full, and the sticky error bits in ERROR_INFO_RPn are still set on a mismatch.

Subordinate side. An analogous pending-ID table exists in AOU_SLV_AXI_INFO, with SFR fields in AXI_SLV_ID_MISMATCH_ERR_RPn (RP0 at 0x0034, RP1 at 0x004C, RP2 at 0x0064, RP3 at 0x007C). However, the subordinate-side mismatch outputs are currently tied to 0 in RTL; the SFR fields are allocated but will always read 0. The aggregate interrupt INT_SIO_ID_MISMATCH will therefore never fire in the current implementation.

LinkReset Error Reporting

When AOU_CORE detects a protocol-level error during activation or deactivation, it asserts O_AOU_REQ_LINKRESET (to FDI) and INT_REQ_LINKRESET (to SW). Both signals share the same net and are the logical OR of the four sub-conditions below. After handling the interrupt, SW must perform a LinkReset sequence.

All LinkReset-related SFR fields reside in LP_LINKRESET (0x0010).

Activate/Deactivate Ack Timeout. After sending an ActivateReq or DeactivateReq message, a counter begins. If the

remote die does not return an Acknowledge within $2^{(ACK_TIME_OUT_VALUE + 18)}$ core clock cycles, the timeout fires. `ACK_TIME_OUT_VALUE` [13:11] is 3 bits wide with a default of 4, giving a default timeout of $2^{22} = \sim 4\text{M}$ cycles. The sticky error bits are `ACT_ACK_ERR` [7] and `DEACT_ACK_ERR` [6], both W1C.

Invalid ACTMSG. The activation FSM (`AOU_ACTIVATION_CTRL`) checks each received activation opcode against the current state. If the opcode is not permitted in the current state, `INVALID_ACTMSG_ERR` [1] is set and the offending 4-bit opcode is captured in `INVALID_ACTMSG_INFO` [5:2]. Clear via W1C.

MsgCredit Timeout. After the local die receives `ActivateAck`, it expects a `CreditGrant` message from the remote die indicating the number of resource planes. If no `CreditGrant` arrives within $2^{(MSGCREDIT_TIME_OUT_VALUE + 18)}$ core clock cycles, `MSGCREDIT_ERR` [0] is set. `MSGCREDIT_TIME_OUT_VALUE` [10:8] defaults to 4. Clear via W1C.

R/B Response Error Debug FIFO

When the manager interface receives an AXI B or R response with a non-OKAY status (`SLVERR` or `DECERR`), `AOU_CORE` captures the error details in a dedicated per-RP FIFO so that the remote die can retrieve them via AXI read.

FIFO structure. The `AOU_ERROR_INFO` module (one instance per RP) implements a 4-entry circular FIFO. Each entry stores:

Field	Width	Description
AXI ID	<code>AXI_ID_WD</code>	ID of the failing transaction
AXI ADDR	<code>AXI_ADDR_WD</code>	Address of the failing transaction
RESP	2 bits	<code>BRESP</code> or <code>RRESP</code> value

Both `BRESP` and `RRESP` errors can be pushed simultaneously (two entries in one cycle). If the FIFO is full, new errors are silently dropped.

Enabling access. Before the remote die can read error information, SW must:

1. Set `AOU_CON0.RPn_ERROR_INFO_ACCESS_EN` [27:24] to 1 for the target RP.
2. Configure the dedicated address window by writing the upper 32 bits to `AXI_ERROR_INFO0_RPn.DEBUG_UPPER_ADDR` and the lower 32 bits to `AXI_ERROR_INFO1_RPn.DEBUG_LOWER_ADDR`.

The `AOU_AXIMUX_1XN_SS` address decoder routes any AXI transaction whose address matches `{DEBUG_UPPER_ADDR, DEBUG_LOWER_ADDR}` to the `AOU_ERROR_INFO` module instead of the normal datapath.

Read protocol. The remote die issues an AXI read (`AR`) to the configured debug address. The module returns the FIFO head entry on `RDATA` (containing the concatenated ID, address, and response fields) with `RLAST` = 1.

Pop protocol. To advance to the next entry, the remote die issues a dummy AXI write (`AW + W`) to the same debug address. The simultaneous `AW` and `W` handshake advances the FIFO read pointer, discarding the head entry. A B response (`OKAY`) is returned.

The typical access sequence is: read the head entry, then pop it, and repeat until no more entries remain.

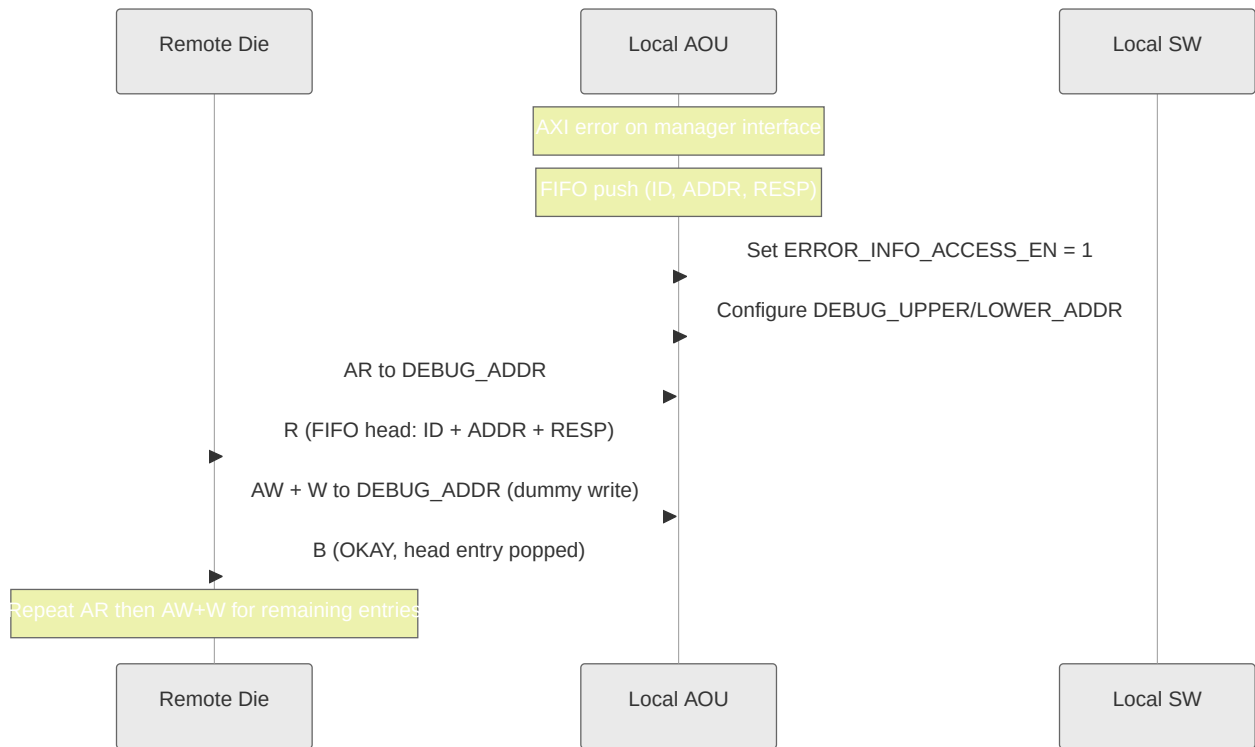


Figure 23. R/B Response Error Debug FIFO Access Sequence

Write Early Response Error

When the early write-response feature is enabled (EARLY_BRESP_EN = 1) and the actual BRESP from the remote die carries an error for a transaction that already received an early OKAY, the error is reported via the INT_EARLY_RESP_ERR interrupt. The failing transaction's AXI ID and BRESP type are captured in WRITE_EARLY_RESPONSE_RPn. SW clears the error by writing 1 to WRITE_RESP_ERR (W1C).

+For a detailed description of the early response error flow, see +ifndef::backend-pdf[] [Transaction Flow -- Error Case](#) +and +SFR Fields. +endif::backend-pdf[] +ifndef::backend-html5[] [Transaction Flow -- Error Case](#) +and +SFR Fields. +endif::backend-html5[]

Activation & Deactivation Flow

The Activation of AOU_CORE begins after the UCIE Link-up process has been successfully completed. The completion of UCIE link-up is indicated by the I_INT_FSM_IN_ACTIVE signal of the AOU_CORE_TOP.

There is no dependency between the activation of the local die and the remote die. Each die may initiate the activation sequence independently, which means the timing of sending **ActivateReq** can differ. Both dies must exchange **ActivateReq** and **ActivateAck** messages to transition to the **ENABLED** state, at which point credited messages can be transmitted.

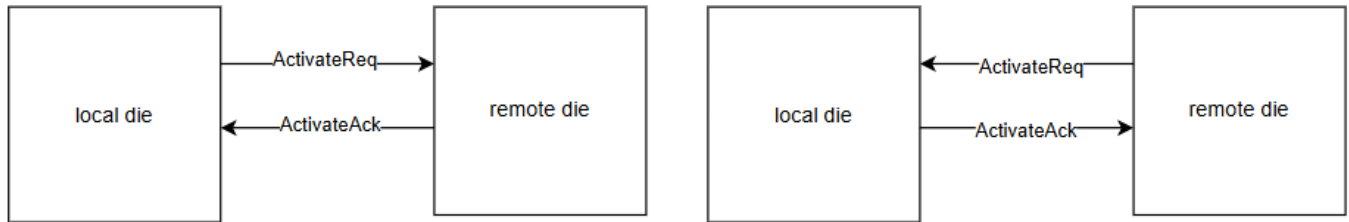


Figure 24. Independent Activation Flow

Regardless of its own activation state, a die must send an ActivateAck in response to an ActivateReq from the other die, to acknowledge receipt of the request.

Credit Management Type

There are two types of Credit Management Type. Since current AOUsPEC cannot resolve pending AXI response without Activate again. When Credit Management type is set to 1, During Deactivate state, AOU_CORE can resolve pending AXI response itself. It can be configured through AOU_CON0.CREDIT_MANAGE, and the default value is 0.

- 0 (default) Based on AoU v0.5
 - After deactivation, if a new request is received from the remote die, no response message can be sent.
 - To deliver the corresponding response, the system must go through activation again after deactivation.
 - Credit management and transmission availability for both Request-related messages and Response-related message are controlled together.
 - Credited messages must not be sent, after the DeactivateReq message is sent.
 - Credits must not be sent after the DeactivateAck message is sent.
 - The Activate Interrupt and Deactivate Interrupt that occur in the process of resuming the exchange of response messages impose mandatory requirement to set Activate and Deactivate SFR.
- 1 Proposal by BOS.
 - When the AoU Activity state is DEACTIVATE, manage Request-related messages(WERQ, RREQ, WDATA) and Response-related messages(RDATA, WRESP) separately.
 - RDATA, WRESP Credited messages can be sent, after the DeactivateReq message is sent.
 - Credits for RDATA, WRESP messages must be sent after the DeactivateAck message is sent.
 - The Deactivate interrupt only provides a hint indicating that the remote die has started activate/deactivate.

Activation start

Activation can be initiated by setting ACTIVATE_START SFR: In the current implementation, credits are granted based on the depth of the RX FIFO, so it is required to ensure that all messages in the RX FIFO have been popped before sending the ActivateReq message.

Activation START(via SFR AOU_INIT.ACTIVATE_START SFR)

- Activation can be triggered by setting ACTIVATE_START.
- Activation does not proceed until I_INT_FSM_IN_ACTIVE is asserted, since no flits can be transmitted beforehand. Therefore, it is allowed to set the ACTIVATE_START SFR before I_INT_FSM_IN_ACTIVE is asserted. If it is set before the assertion, the activation process will automatically proceed after I_INT_FSM_IN_ACTIVE becomes asserted.

- AOU_INIT.ACTIVATE_START SFR is automatically cleared to 0 once the activation is completed and AOU_ACTIVATE_STATE transitions to ENABLED.

Deactivation start

Deactivation is initiated by setting the AOU_INIT.DEACTIVATE_START register. Setting this register does not immediately trigger sending a DeactivateReq message. The detailed conditions and sequence for deactivation can be found in the **Deactivation Sequence Flow** section. This section will only describe AOU_INIT.DEACTIVATE_TIME_OUT_VALUE SFR.

- When software initiates a DeactivateReq by setting the SFR, there may still be outstanding requests that have not yet reached out AOU_CORE.
- To handle this safely, a timeout mechanism is implemented to ensure that no valid packets remain in the AOU_TX_CORE.
- If software guarantees that all responses to its issued requests have been received before setting the deactivation start SFR, the deactivate TIME_OUT_VALUE(AOU_INIT.DEACTIVATE_TIME_OUT_VALUE SFR) can be safely configured to a shorter duration.

The ACTIVATION_OP field encodes deactivation messages as follows:

- 2 = DeactivateReq
- 3 = DeactivateAck

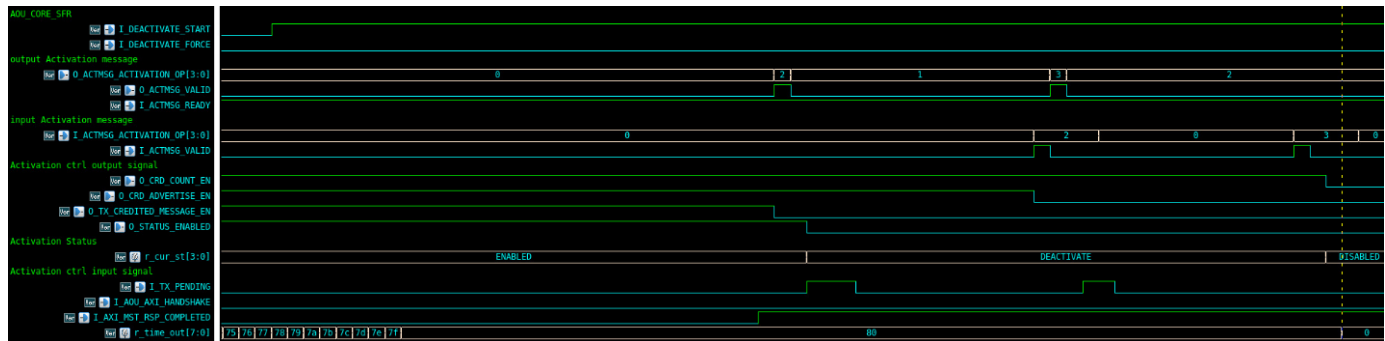


Figure 25. Deactivation sequence with DEACTIVATE_START

Deactivation Sequence Flow

Once the local die issues a DeactivateReq, it can no longer provide responses to transactions initiated by the remote die. If the remote die continues to send requests or waits for responses without being notified, it may enter a hang state. To manage this safely, the remote die must take explicit action upon receiving a DeactivateReq: 1. Immediately generate an interrupt to the CPU to inform the system software that a DeactivateReq has been received and that it must set the DEACTIVATE_START SFR.

Although deactivation of the local die and the remote die operate independently, it is essential at the system level to communicate the deactivation & activation state through interrupts. This ensures that system software is explicitly informed of deactivation events and prevents the remote die from continuing to expect response that will never arrive, thereby avoiding hang conditions. This approach guarantees that once deactivation is initiated, both dies can coordinate the transition into safe and consistent DISABLED state.

If the software on both dies can explicitly coordinate to guarantee that all outstanding transactions have been completed and that no new transactions will be issued, then such an complicated implementation would not be necessary.

Figure 25 illustrates the current activate/deactivate implementation of AOU_CORE.

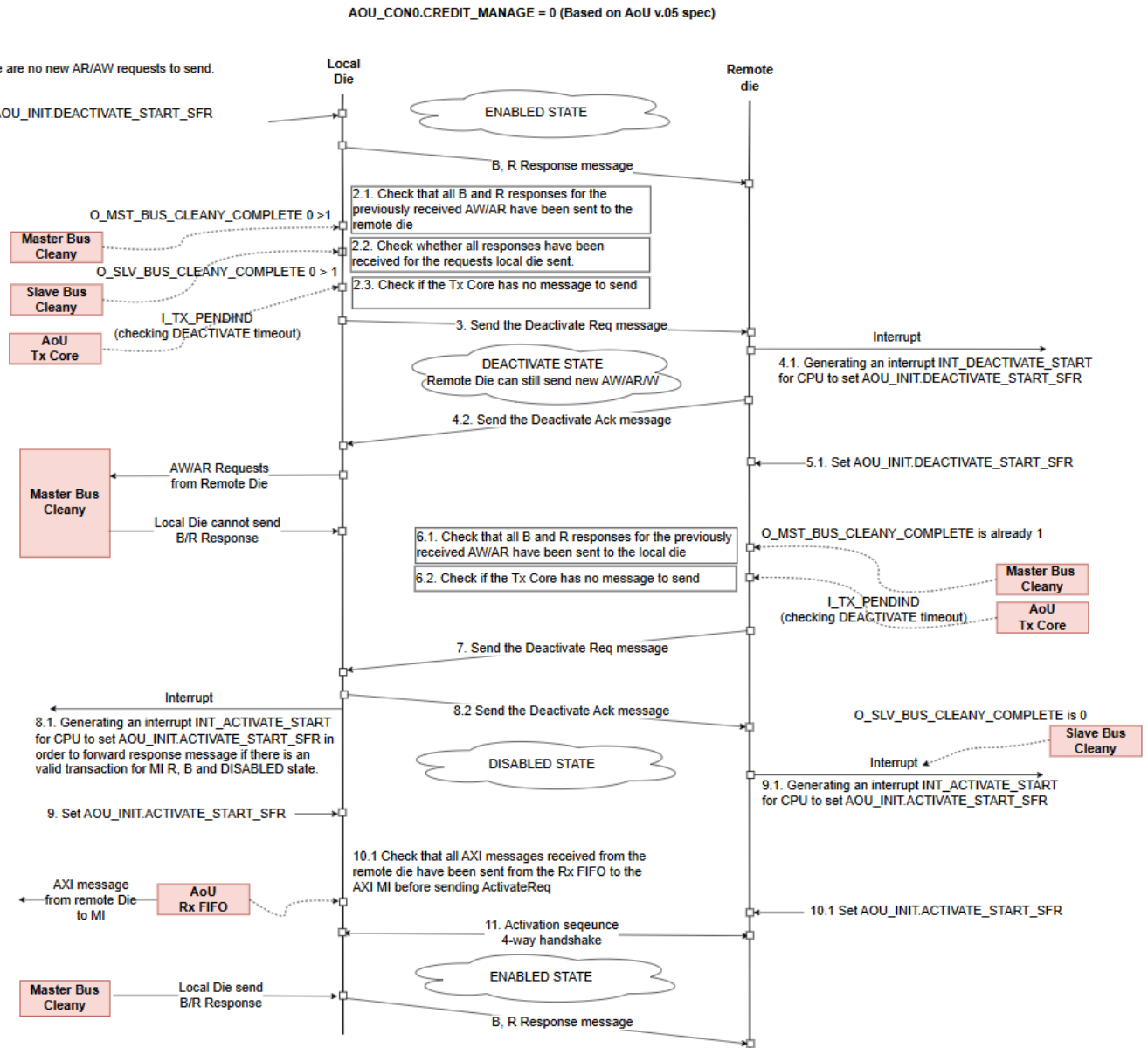


Figure 26. CREDIT_MANAGE 1 Deactivate sequence flow

AOU_CON0.CREDIT_MANAGE = 1 (Proposal)

Participants: Local Die, Remote Die, Master Bus Cleany, Slave Bus Cleany, AoU Tx Core.

Sequence of Events:

- 1.1. There are no new AR/AW requests to send.
- 1.2. Set AOU_INIT.DEACTIVATE_START_SFR.
 - Remote die send B/R Response to Master Bus Cleany.
- 2.1. Check that all previously issued AW/AR requests have received B/R responses.
- 2.2. Check if the Tx Core has no message to send.
 - Slave Bus Cleany sends I_TX_PENDING to Local Die.
 - AoU Tx Core sends I_TX_PENDING to Local Die.
3. Send the Deactivate Req message to Remote Die.
- 4.1. Generating an interrupt INT_DEACTIVATE_START for CPU to set AOU_INIT.DEACTIVATE_START_SFR.
 - Interrupt signal from Remote Die to CPU.
- 4.2. B, R Credits for Local die can still be granted.
- 4.3. Send the Deactivate Ack message to Remote Die.
- 5.1. There are no new AR/AW requests to send.
- 5.2. Set AOU_INIT.DEACTIVATE_START_SFR.
- 6.1. Check that all previously issued AW/AR requests have received B/R responses.
- 6.2. Check if the Tx Core has no message to send.
 - Slave Bus Cleany sends I_TX_PENDING to Remote Die.
 - AoU Tx Core sends I_TX_PENDING to Remote Die.
7. Send the Deactivate Req message to Local Die.
8. Send the Deactivate Ack message to Local Die.

State Transitions:

- ENABLED STATE** (Local Die) to **DEACTIVATE STATE** (Remote Die) via Step 3.
- DEACTIVATE STATE** (Remote Die) to **DEACTIVATE STATE** (Local Die) via Step 4.3.
- DEACTIVATE STATE** (Local Die) to **DISABLED STATE** (Local Die) via Step 8.

Additional Notes:

- Master Bus Cleany: AW/AR Requests from Remote Die; Local Die can still send B/R Response (Violating the AoU Spec v0.5).
- O_SLV_BUS_CLEANY_COMPLETE 0 > 1 (Slave Bus Cleany).
- Ucle begins the state transition process on FDI and RDI.

Activation/Deactivation Interrupt

INT_ACTIVATE_START

- When this interrupt is detected, AOU_INIT.ACTIVATE_START SFR must be set to 1.
- When AOU_INIT.ACTIVATE_STATE_ENABLED becomes 1, write 1 to clear the AOU_INIT.INT_ACTIVATE_START W1C SFR.
- An interrupt occurs when AOU_INIT.ACTIVATE_START SFR is not set and any of the following conditions are met:
 1. There is a message to send
 - When AOU_CON0.CREDIT_MANAGE = 0, if a new request arrives from the remote die after the local die has sent a DeactivateReq, this interrupt is asserted because the local die must return a response.
 - When AOU_CON0.CREDIT_MANAGE = 1, this interrupt can also be asserted if a new request arrives after the remote die has sent a DeactivateReq.
 2. There is a response to be received (I_SLV_BUS_CLEANY_COMPLETE == 0)
 - When AOU_CON0.CREDIT_MANAGE = 0, this interrupt can be asserted if the local die issues new AXI requests after the remote die has sent a DeactivateReq.
 3. An ActivateReq message is received from the remote die.

INT_DEACTIVATE_START

- When AOU_CON0.CREDIT_MANAGE = 0: if the interrupt is detected, the manager IP must stop sending new request messages and AOU_INIT.DEACTIVATE_START SFR must be set to 1.
- When AOU_CON0.CREDIT_MANAGE = 1: if the interrupt is detected, this serves only as a hint that the remote die intends to deactivate. The local die can continue sending request messages. Once all messages have been sent, AOU_INIT.DEACTIVATE_START SFR must be set. Otherwise, the remote die may end up in a state where it can never send requests again.
- When AOU_INIT.ACTIVATE_STATE_DISABLED becomes 1, write 1 to clear the AOU_INIT.INT_DEACTIVATE_START W1C SFR.
- An interrupt is asserted when AOU_INIT.DEACTIVATE_START SFR is not set and the local die receives a DeactivateReq message from the remote die.

PM entry / LinkReset / LinkDisabled sequence

For PM entry / LinkReset / LinkDisable entry sequence, UCIE_CORE check AOU_CORE state and try to change state. For this sequence, resolving pending AXI transactions is necessary.

Since current AOU SPEC has no way to send AXI response after sending DeactiveReq. CREDIT_MANAGE = 0 (Type 0) is matched with current AOU_SPEC. For this case, SW need to check whether there is pending AXI transaction.

• PM entry SW sequence

1. Write AOU_INIT.DEACTIVATE_START to 1.
2. Polling AOU_INIT.ACTIVATE_STATE_DISABLED and AOU_INIT.SLV_TR_COMPLETE & AOU_INIT.MST_TR_COMPLETE.
 - a. Although AOU_CORE state becomes DISABLED, there may have pending AXI transaction.
 - b. AOU_CORE issue Interrupt for resolving pending AXI transactions.

- c. Activate AOU_CORE and resolve pending AXI transactions.
- d. Deactivate AOU_CORE
3. Polling AOU_INIT.ACTIVATE_STATE_DISABLED and AOU_INIT.SLV_TR_COMPLETE & AOU_INIT.MST_TR_COMPLETE.
4. Do PM Entry Sequence on UCIE_CORE.

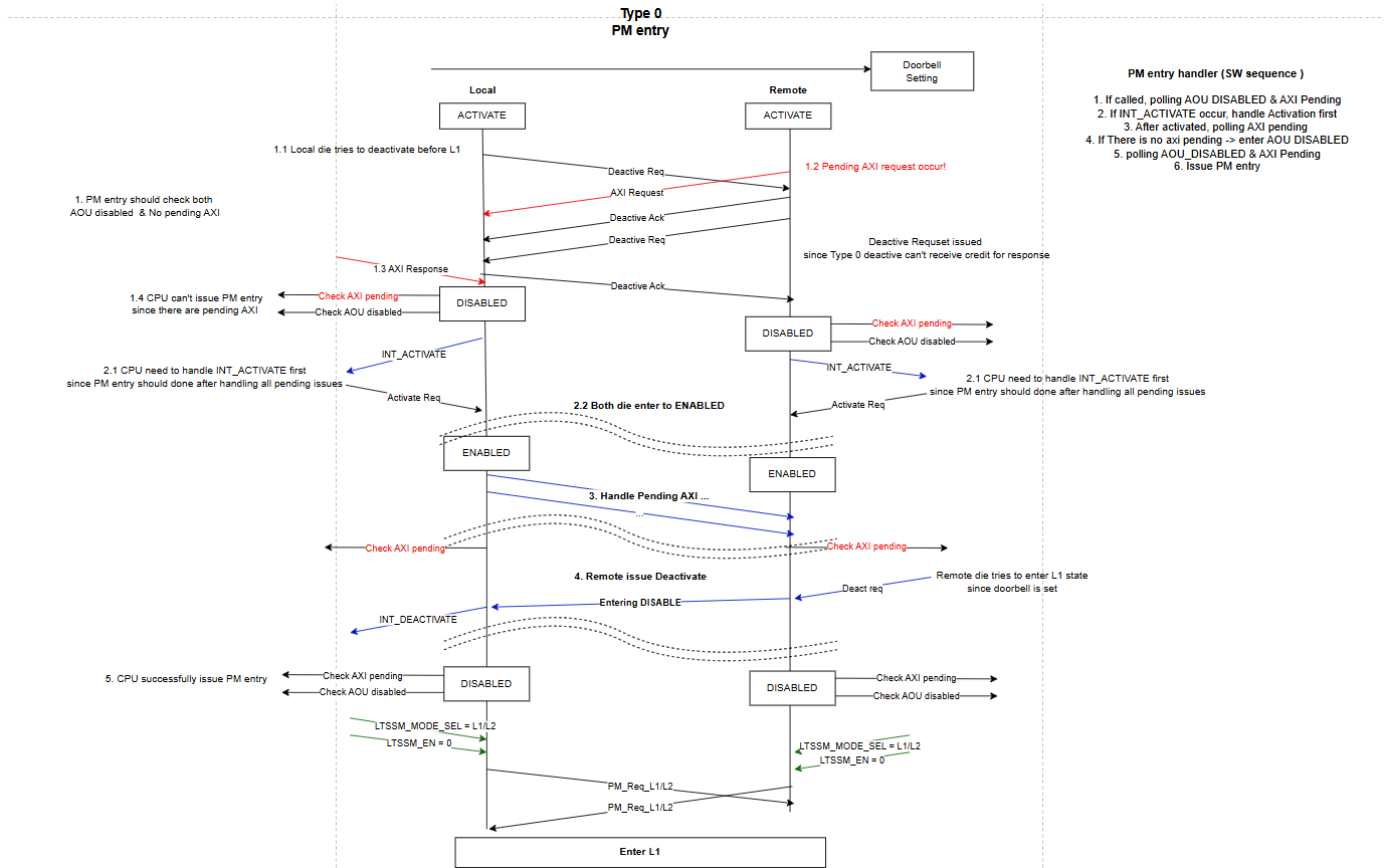


Figure 28. PM Entry Sequence

• LinkDisable SW sequence

Same as PM entry, before doing UCIE state transition, AOU_CORE need to Disabled properly.

1. Write AOU_INIT.DEACTIVATE_START to 1.
2. Polling AOU_INIT.ACTIVATE_STATE_DISABLED and AOU_INIT.SLV_TR_COMPLETE & AOU_INIT.MST_TR_COMPLETE.
 - a. Although AOU_CORE state becomes DISABLED, there may have pending AXI transaction.
 - b. AOU_CORE issue Interrupt for resolving pending AXI transactions.
 - c. Activate AOU_CORE and resolve pending AXI transactions.
 - d. Deactivate AOU_CORE
3. Polling AOU_INIT.ACTIVATE_STATE_DISABLED and AOU_INIT.SLV_TR_COMPLETE & AOU_INIT.MST_TR_COMPLETE.
4. Do LinkReset/LinkDisabled Sequence on UCIE_CORE.

◦ LinkReset sequence

If AOU_CORE faces uncorrectable error (ex. AOU SPEC violation), AOU_CORE sends LinkReset to CPU and D2D adapter.

LinkReset indicates that an error has occurred which requires the Link to go down.

While handling LinkReset, SW needs to do AOU_CORE SW reset by setting AOU_CON0.AOU_SW_RESET.